



多周期CPU设计

2025年秋

本讲提要

- 单周期CPU特点
- 单周期CPU主要不足
- 多周期CPU设计
 - 多周期CPU控制器基本组成
 - 多周期CPU设计

控制器的功能

□ 冯. 诺依曼结构的计算机

- “存储程序”计算机，设置内存，存放程序和数据
- 在程序运行之前将程序调入内存，然后执行程序

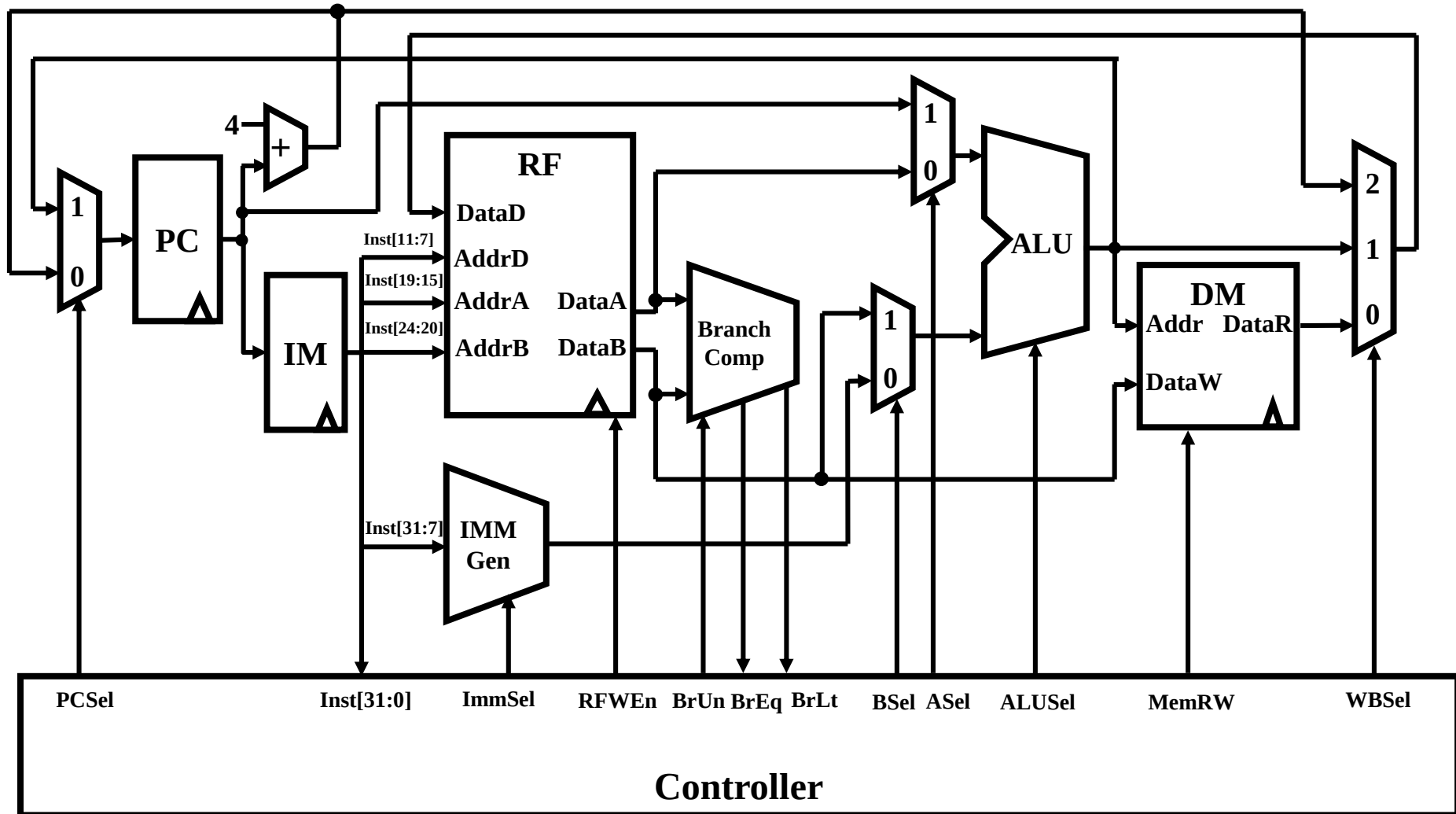
□ 计算机的功能是执行程序

- 程序是依次排列起来的指令序列

□ 计算机执行程序的基本过程

- 从程序首地址开始执行第一条指令
- (分步)执行每一条指令，并形成下一条待执行指令地址
- 自动地连续执行指令，直到程序的最后一条指令

完整的单周期CPU



单周期CPU特点

□ 优点

- 每条指令占用一个CPU周期
- 逻辑设计简单，时序设计也简单

□ 缺点

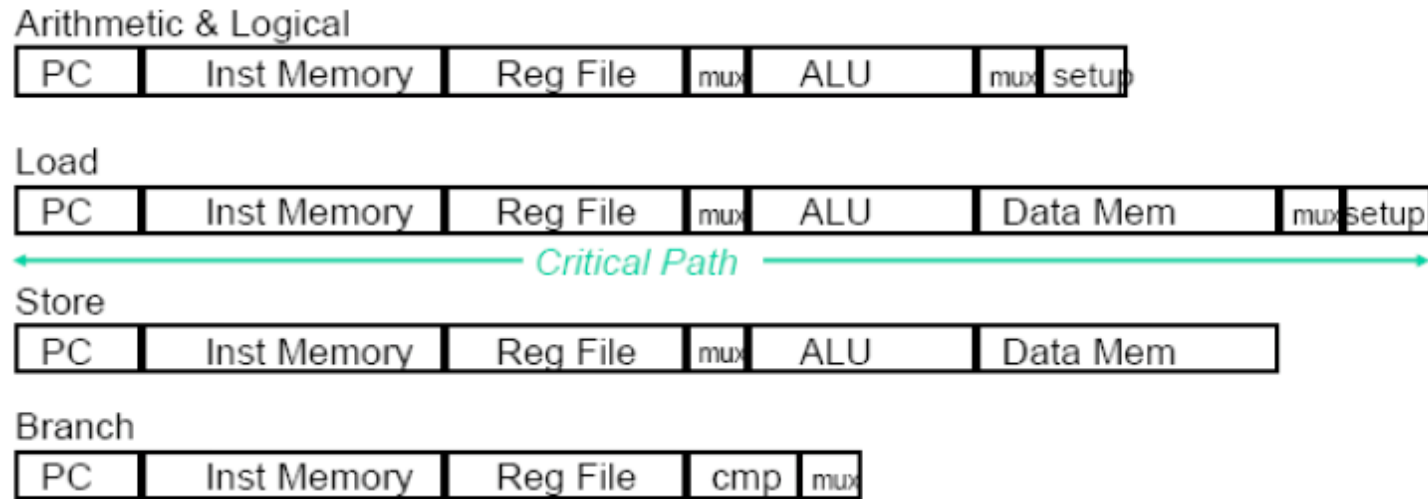
- 各组成部件的利用率不高
 - 各部件大部分时间在保持信号
- 时钟周期需要满足执行时间最长指令的要求
- Load指令
- $CPI = 1$

单周期CPU性能

- 假定某单周期CPU各主要部件的延迟为：
- 存储器（Memory）：2ns
- 运算器（ALU/Adder）：2ns
- 寄存器组（Register File）：1ns

Inst.	Inst. Mem	Reg. Read	ALU	Data Mem	Reg. Write	Total
ALU	2	1	2		1	6
lw	2	1	2	2	1	8
sw	2	1	2	2		7
br	2	1	2			5

单周期CPU的性能



- ❑ 指令周期比较长
- ❑ 所有指令都必须使用最长的周期

单周期CPU性能

□ 该单周期CPU，执行100条指令：

- 25%的Load指令
- 10%的Store指令
- 45%的算逻指令
- 20%的跳转指令

□ 单周期的执行时间

- $100 \times 8 = 800\text{ns}$

□ 可能的优化

- $25 \times 8 + 10 \times 7 + 45 \times 6 + 20 \times 5 = 640\text{ns}$
- $\text{Speedup} = 800 / 640 = 1.25$

多周期CPU

□ 将指令执行过程分解成多个步骤

- 和单周期CPU基本相同
- 每条指令占用它需要的步骤数

□ 每个步骤占用一个时钟周期

- 尽量平衡各步骤间的延迟
- 尽量限制每个步骤使用单一的主要部件
- 控制器仅需提供当前步骤所需要的控制信号

□ 前提

- 保存好下一步骤要用到的值
 - 引入“新”的内部寄存器
- 转到下一步骤执行
 - 引入状态标记当前步骤
 - 有限自动机

多周期CPU的控制器

□ 控制器的功能就是控制指令的执行过程

- 能够正确并且自动地连续执行指令
 - 按程序中设定的指令次序执行
- 正确地分步完成每一条指令规定的功能
 - 读取指令→ 分析指令→ 执行指令

□ 进一步讲，就是向计算机各功能部件提供协调运行每一个步骤所需要的控制信号

控制器的组成

□ ①程序计数器PC

- 存放指令地址，有增量或接收新值功能

□ ②指令寄存器IR

- 存放指令内容：操作码与操作数地址

□ ③指令执行步骤标记线路

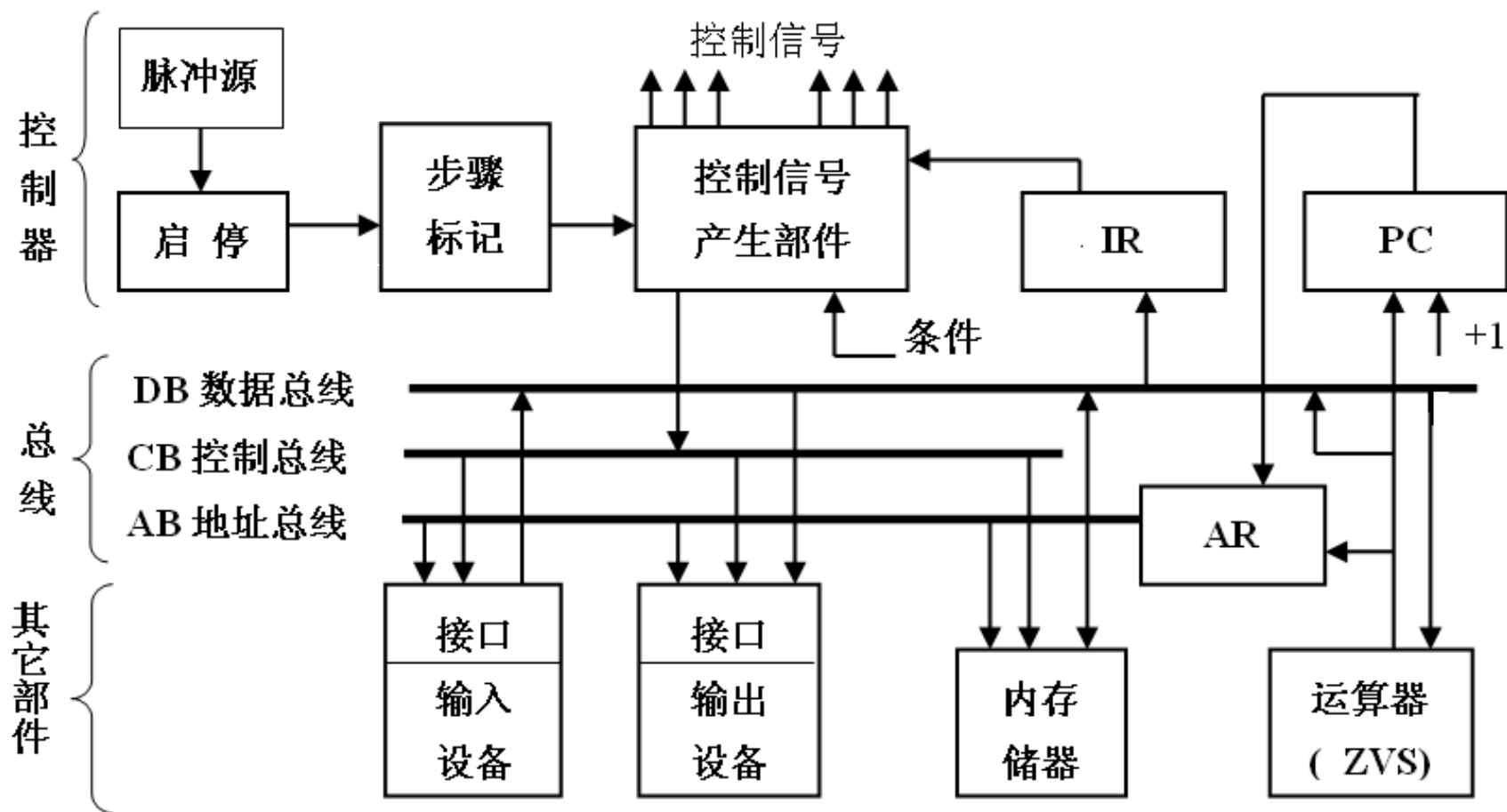
- 指明每条指令的执行步骤和相对次序关系

□ ④控制信号产生线路

- 给出计算机各功能部件协同运行所需要的控制信号

□ 主脉冲源与启停控制线路

多周期CPU控制器组成



两种不同类型的控制器

□ 根据指令步骤标记线路和控制信号产生线路不同的组成和不同的运行原理，有两种不同类型的控制器：

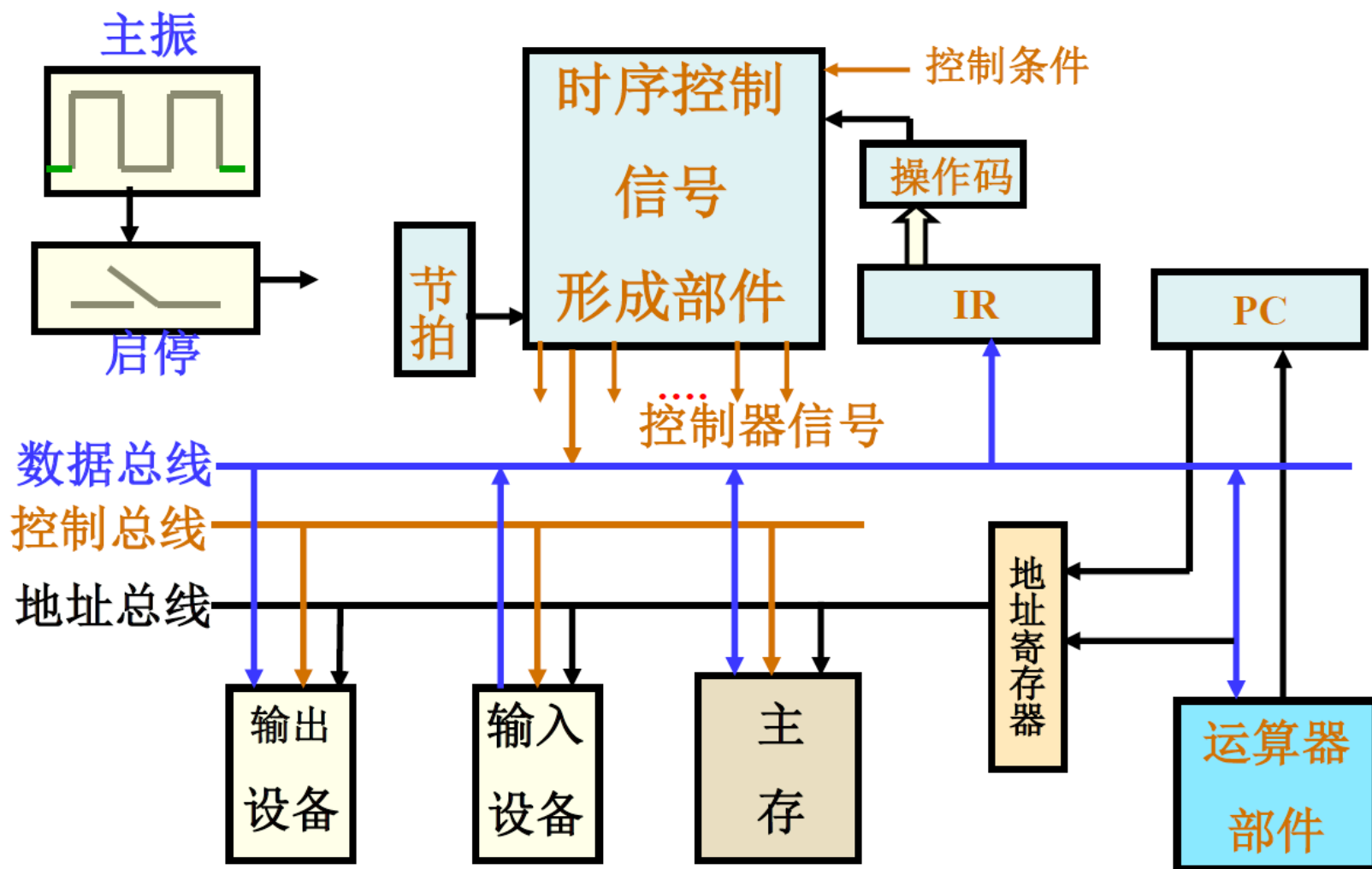
□ 硬连线控制器(组合逻辑控制器)：

- 采用组合逻辑线路、依据指令及其执行步骤直接产生控制信号。

□ 微程序控制器：

- 采用存储器电路把控制信号存储起来，依据指令执行的步骤读出要用到的信号组合。

硬连线控制器



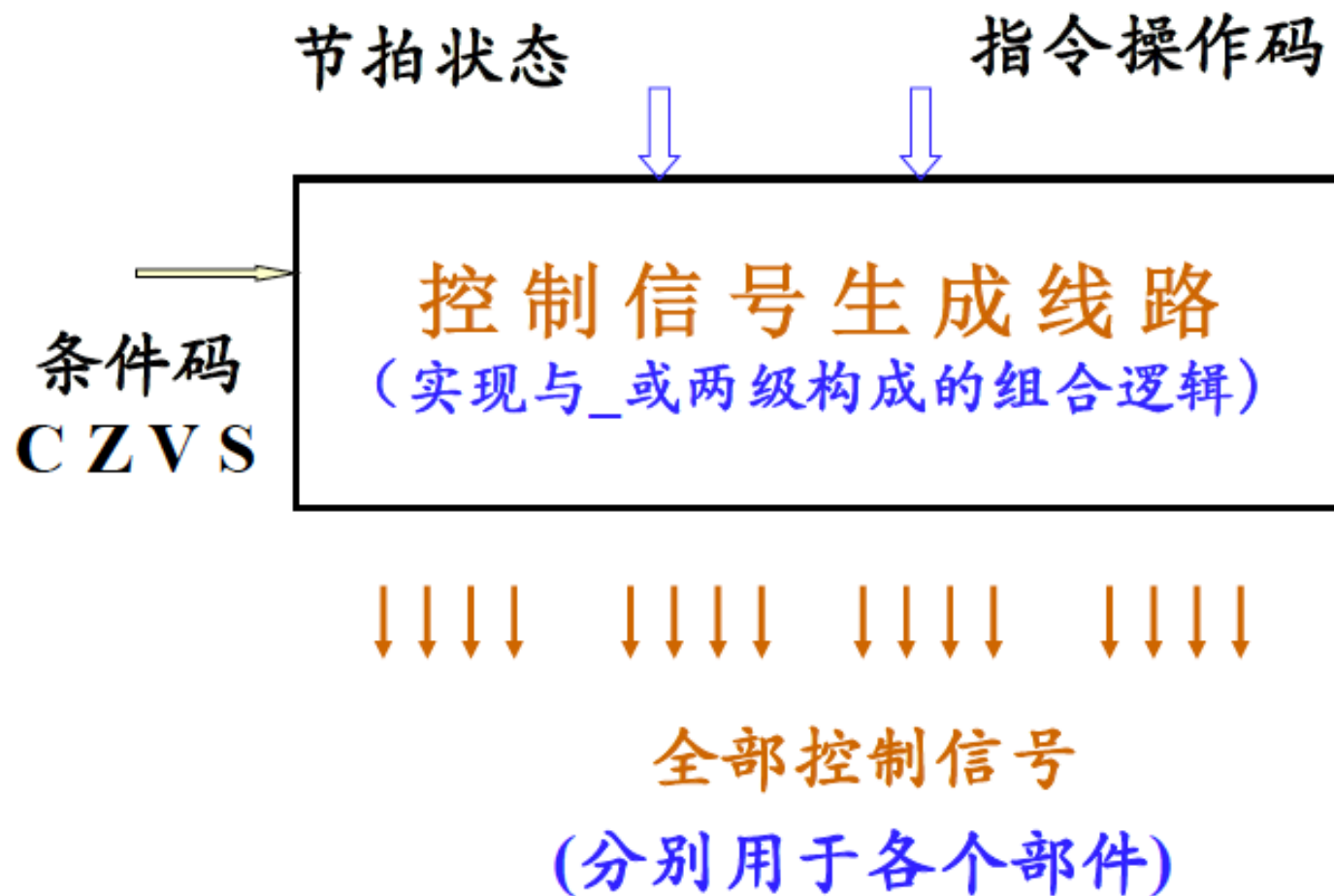
硬连线控制器组成与实现

- ❑ 硬连线控制器由程序计数器PC、指令寄存器IR、节拍发生器Timer和控制信号产生部件4部分组成。
- ❑ PC用于提供待读出指令在主存储器中的地址，
- ❑ IR用于保存从主存储器中读出的指令内容，
- ❑ Timer 用于给出并维护指令执行步骤的编码，
- ❑ 控制信号产生部件用于依据指令内容（在IR中）和指令执行所处的操作步骤（Timer 提供），用组合逻辑线路产生计算机本操作步骤中各个部件所需要的控制信号。
- ❑ 划分指令执行步骤，确定各步骤应执行的功能和步骤之间的衔接关系，以及确定各部件完成这些功能所需要的控制信号，是控制器设计的几个关键环节。

硬连线控制器组成与实现

- ❑ 在多周期CPU 系统中, 要按照指令总的功能要求, 把不同的功能序列划分到相应的步骤, 再落实到不同的部件, 控制器需要按照**指令及其执行步骤**, 为计算机各个部件提供它们协同运行所需要的控制信号。
- ❑ 向各部件提供哪些控制信号, 决定于各部件的运行要求。为此必须规划汇总各部件在各个执行步骤中要求使用的控制信号。**这些信号是用组合逻辑电路产生的, 可以表示为: 信号 $i = f$ (指令内容, 执行步骤等), 通常表现为多个由与、或两级逻辑构成的表达式。**

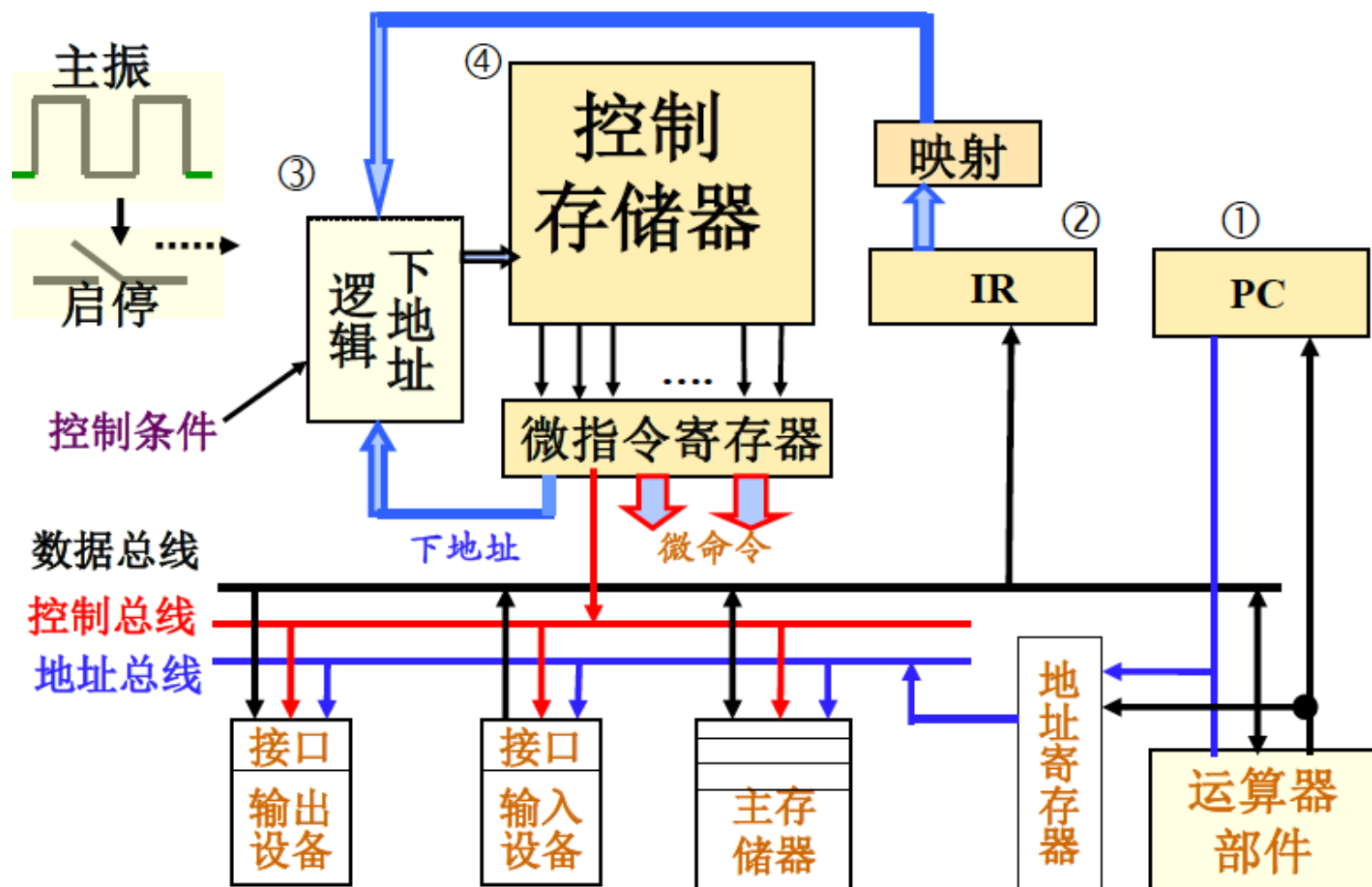
控制信号生成线路



微程序控制器

- 采用控制存储器存储每条指令的每个执行步骤所需要的全部控制信号
 - 用微地址进行访问，读出控制信号并输出
- 采用下地址逻辑实现执行步骤之间的衔接
 - 根据指令操作码映射出该指令的首条微指令的地址
 - 每条微指令给出其下一步骤的微地址

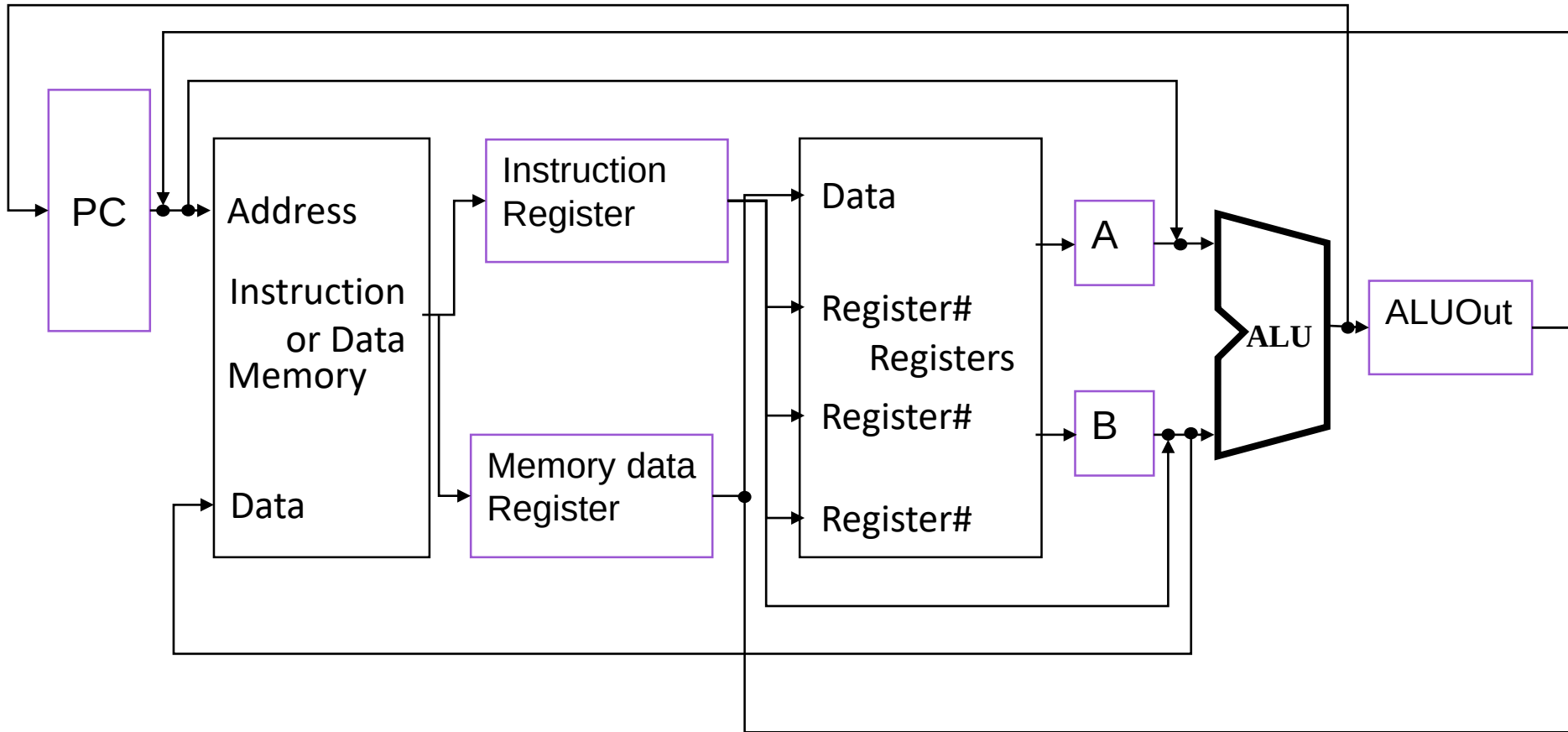
微程序控制器的组成



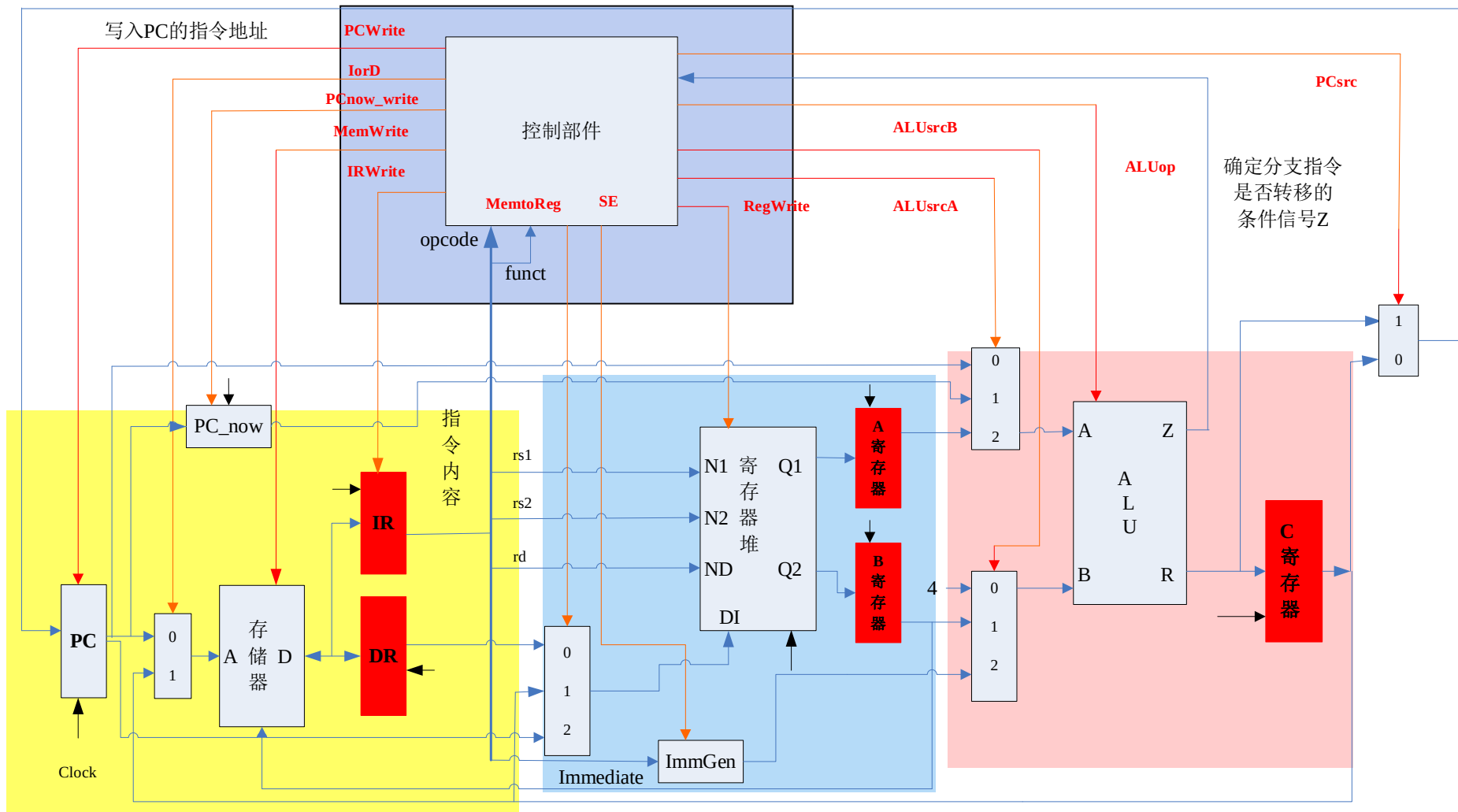
多周期的CPU控制器设计

- 确定数据通路
- 划分指令执行步骤
 - 指令流程图
- 安排每条指令每个步骤的功能,并给出相应的控制信号
 - 指令流程表
- 为指令执行步骤设计状态机
- 为每个步骤的控制信号设计控制信号生成逻辑

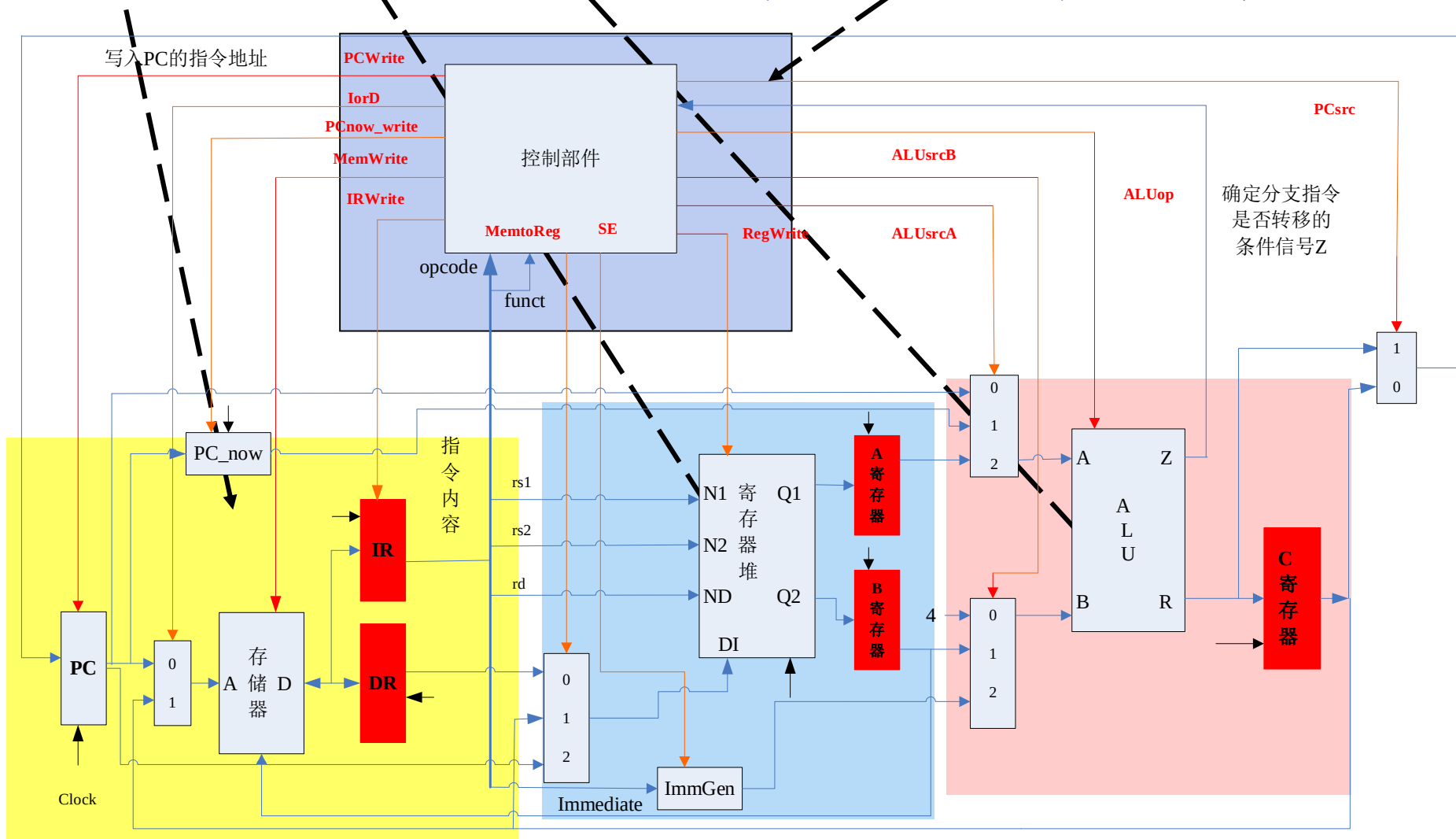
多周期CPU的Datapath



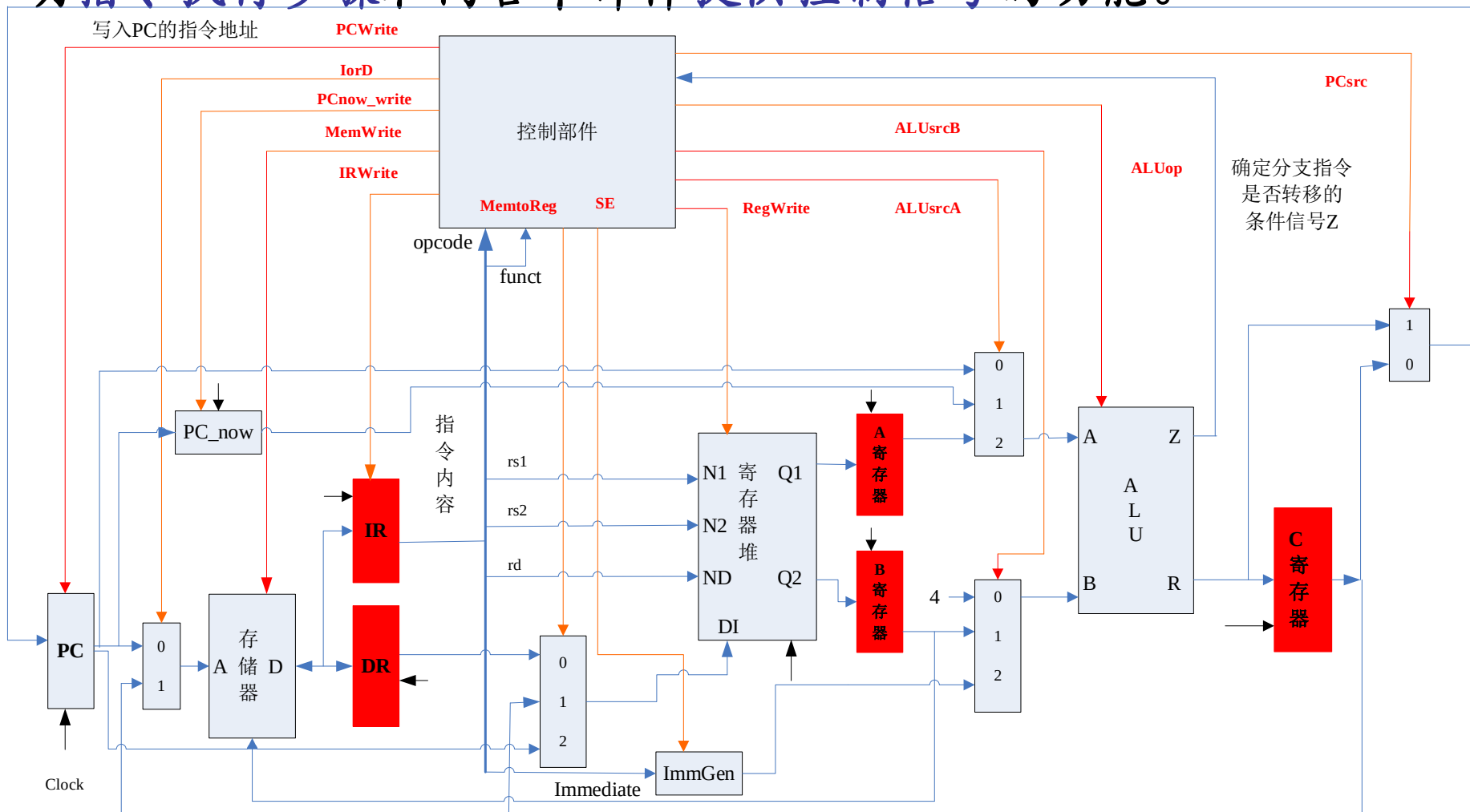
多周期计算机硬件系统组成



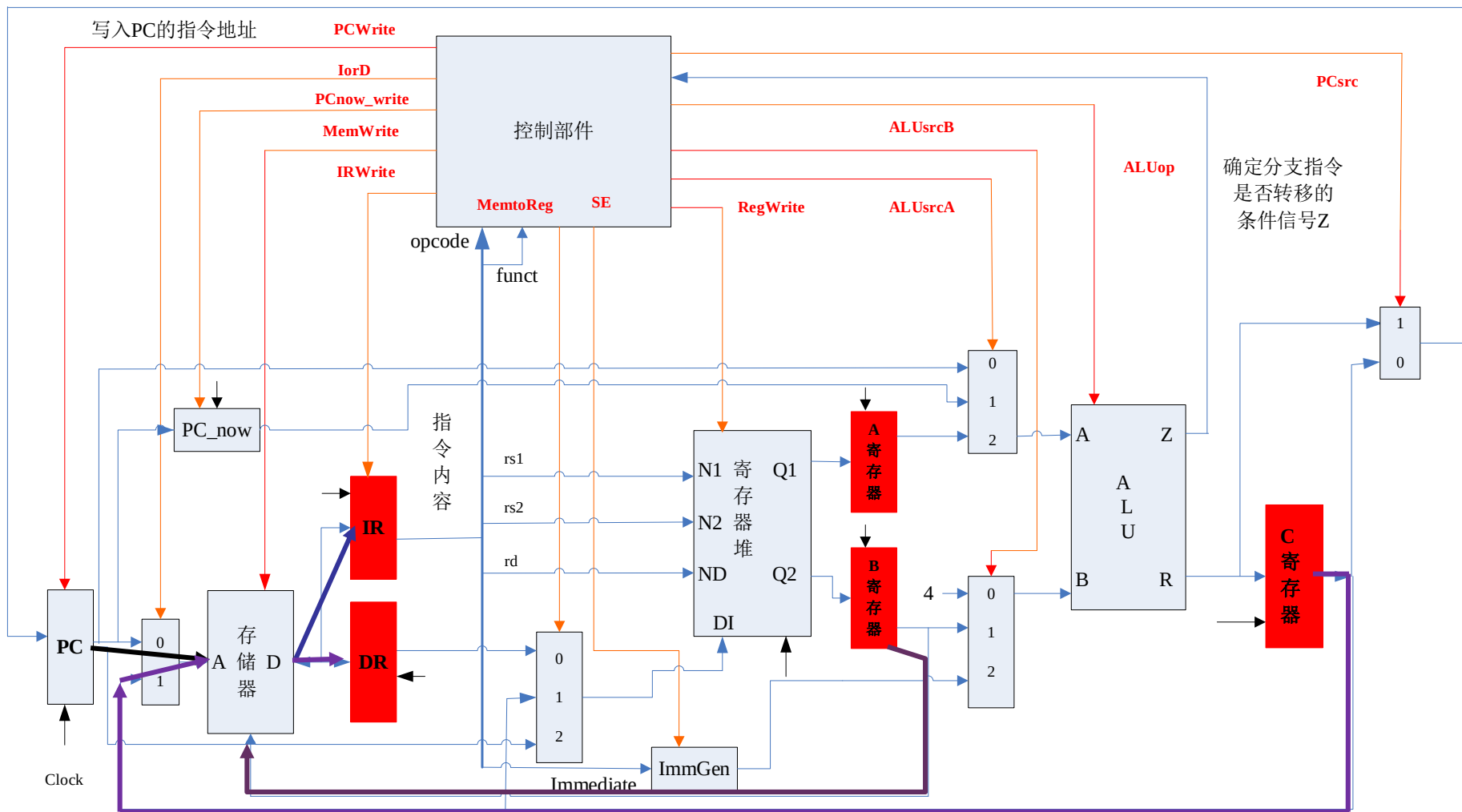
由存储器、寄存器堆、ALU部件、控制部件 4部分组成



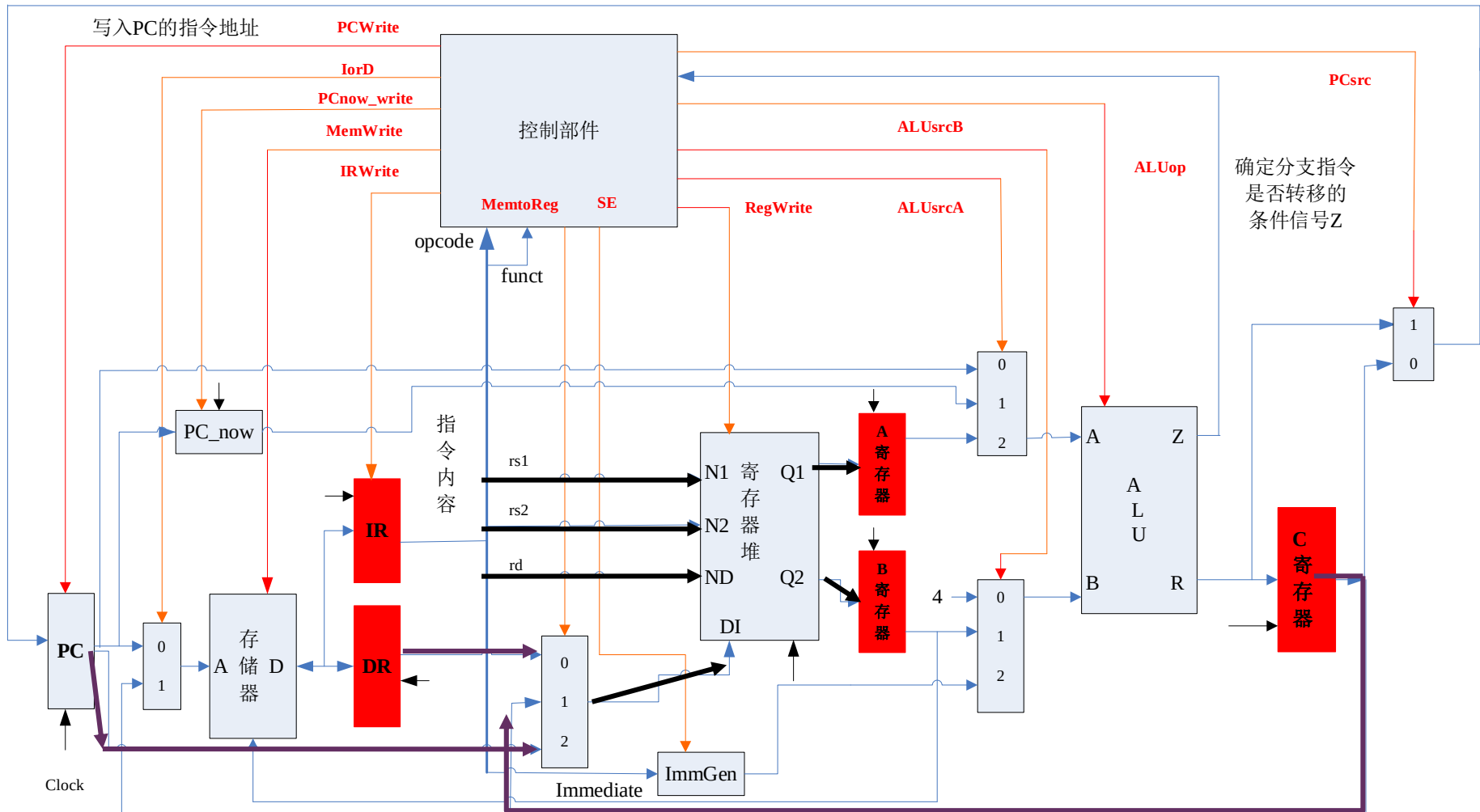
控制部件由节拍发生器和控制信号产生线路组成，分别完成标明指令执行步骤和向各个部件提供控制信号的功能。



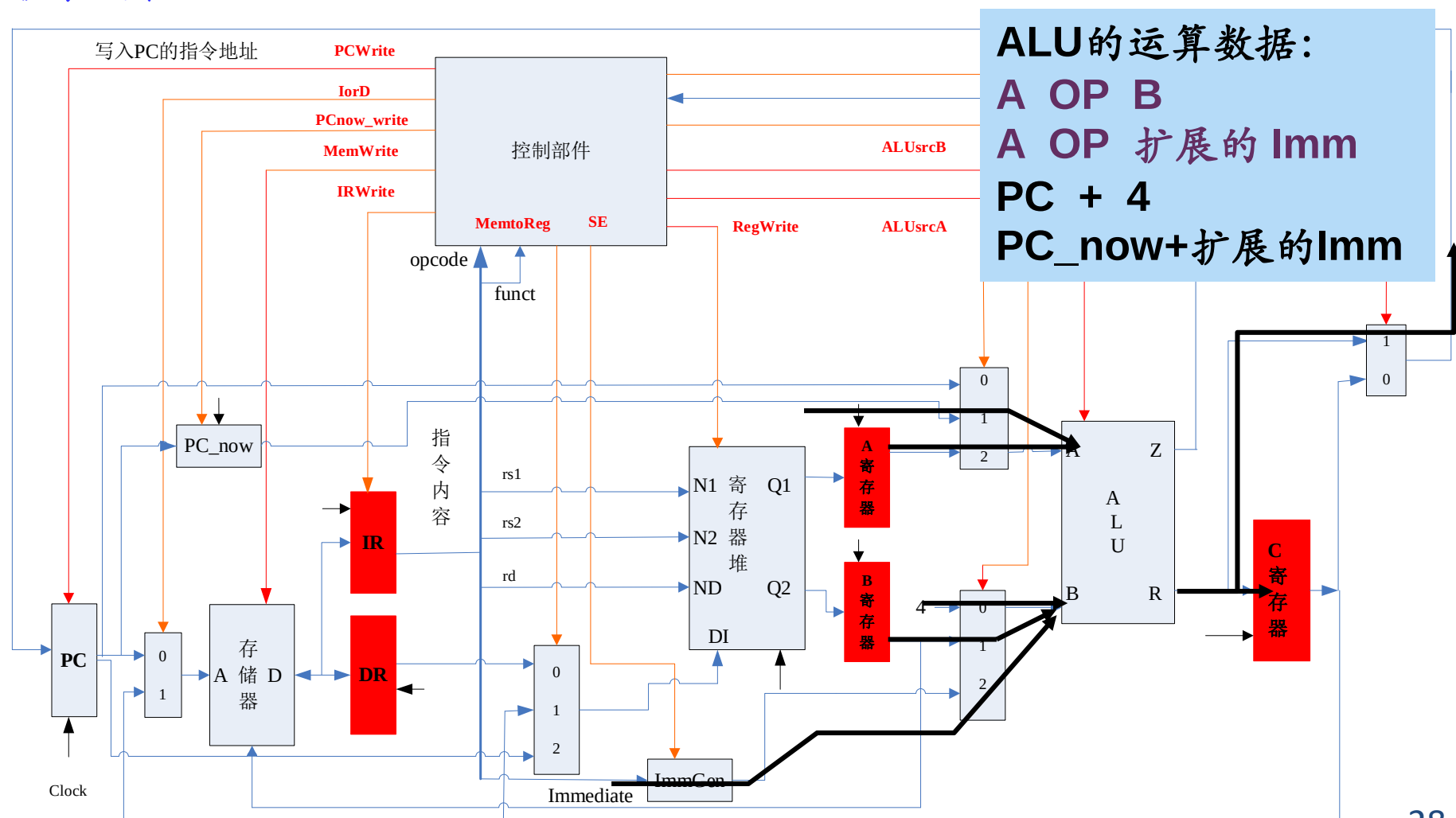
存储器存指令和数据。读指令时由PC提供地址，读出的指令保存到IR；读写数据时由C寄存器提供地址，读操作的读出数据保存到DR；写操作的写入数据由B寄存器给出。



寄存器堆由32个寄存器组成，可以用N1(rs1)、N2(rs2)同时读出两个寄存器的内容，分别存于A、B寄存器；可以用ND(rd)把DI端的数据写入；
被写入数据来自C寄存器、DR或PC。



ALU完成算术和逻辑运算，两路输入分别为A和B，其中A路输入可选择A寄存器、PC或PC_now，B路输入可选择B寄存器或常数4、IR.immediate经适当扩展的值。



指令执行步骤

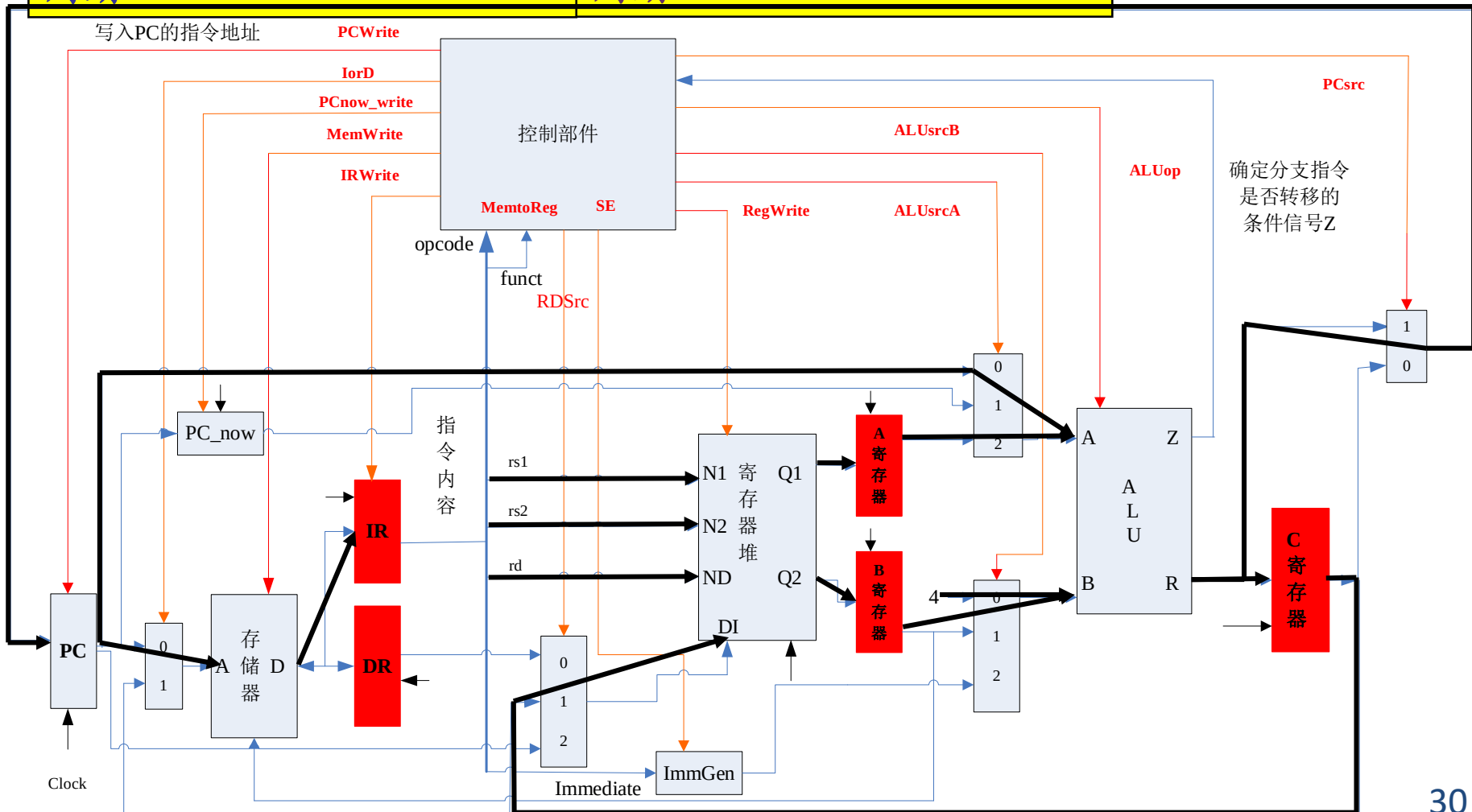
31	30	25	24	21	20	19	15	14	12	11	8	7	6	0				
funct7				rs2			rs1		funct3		rd			opcode		R-type		
imm[11:0]						rs1		funct3		rd			opcode		I-type			
imm[11:5]				rs2			rs1		funct3		imm[4:0]			opcode		S-type		
imm[12]		imm[10:5]			rs2			rs1		funct3		imm[4:1]		imm[11]		opcode		B-type
imm[31:12]									rd			opcode			U-type			
imm[20]		imm[10:1]				imm[11]		imm[19:12]			rd			opcode		J-type		

- ❑ **RISC-V**的取指操作可**1**步（节拍）完成，在取指之后：
- ❑ **B**型指令：读寄存器堆(**RF**)、**ALU**运算(**EXE**)，可**2**步完成，
- ❑ **R\J\UI**和**S**型指令：读寄存器堆(**RF**)、**ALU**运算(**EXE**)和结果写回(**WB**)，可**3**步完成，
- ❑ **Load**指令读内存指令：读寄存器堆(**RF**)、**ALU**算地址(**EXE**)、读内存数据到**DataR(MEM)**，把**DataR**内容写入寄存器堆(**RF**)，可**4**步完成。

RISC-V 的 R 型 ADD 指令的执行过程

ADD
R-type

取指 周期	$IR \leftarrow MEM[PC]$ $PC \leftarrow PC+4$	译码 周期	$A \leftarrow [rs1]$ $B \leftarrow [rs2]$
执行 周期	$C \leftarrow A + B$	写回 周期	寄存器堆[rd] $\leftarrow C$



R型指令的实现(ADD)

□ 取指令

- IorD=0, ALUsrcA=0, ALUsrcB=0, ALUop='ADD', PCsrc=1
- MEMwrite=0, IRwrite=1, PCwrite=1, PCnow_write=1

□ 译码/取操作数

- AEn=1, BEn=1
- MEMwrite=0, IRwrite=0, PCwrite=0, PCnow_write=0

□ 执行运算

- ALUsrcA=0, ALUsrcB=0, ALUop='ADD'

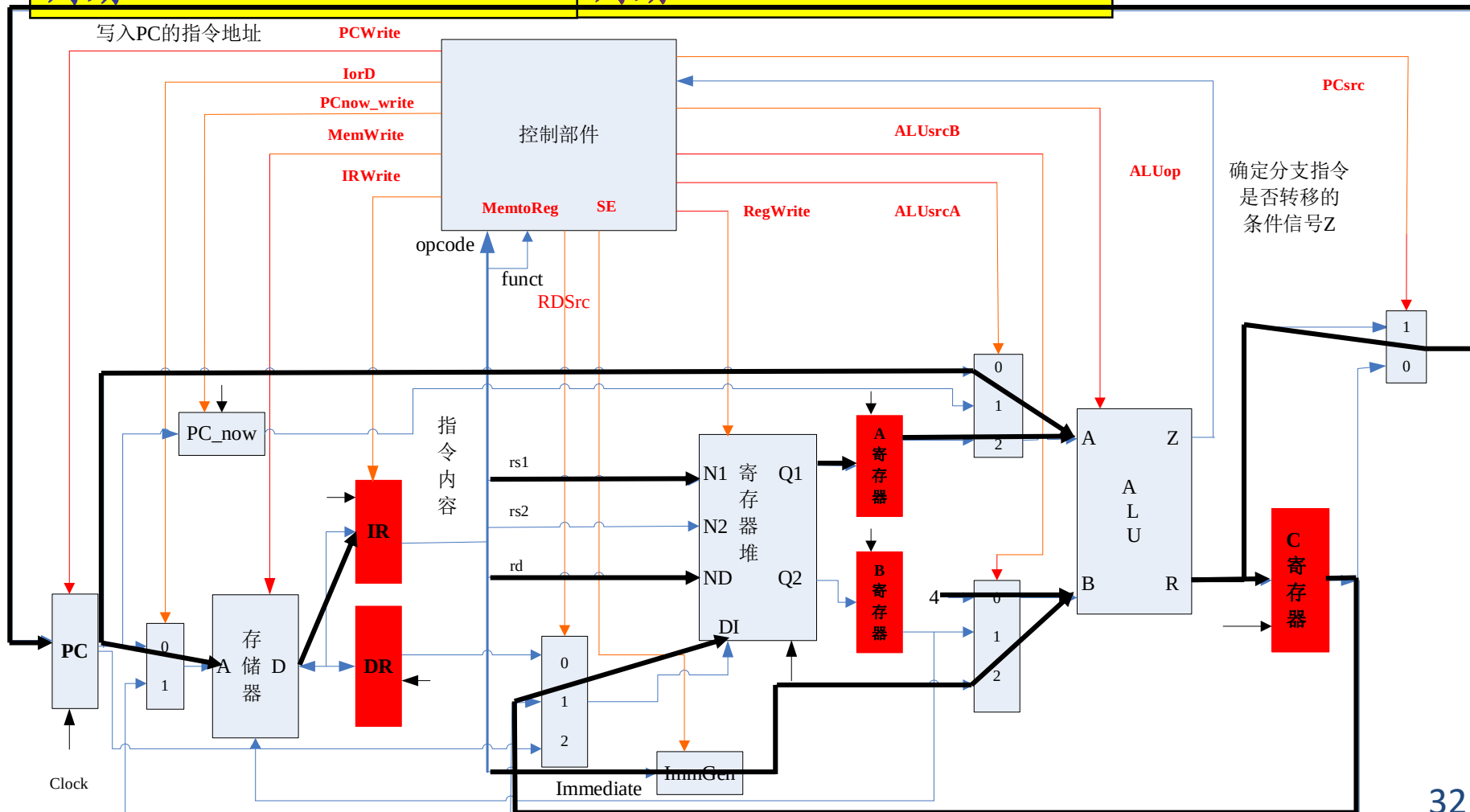
□ 写回寄存器

- RegWrite=1, RDSrc=1

RISC-V 的 I 型 ADDI 指令的执行过程

ADDI
I-type

取指 周期	$IR \leftarrow MEM[PC]$ $PC \leftarrow PC+4$	译码 周期	$A \leftarrow [rs1]$
执行 周期	$C \leftarrow A + \text{扩展 Imm}$	写回 周期	寄存器堆[rd] $\leftarrow C$

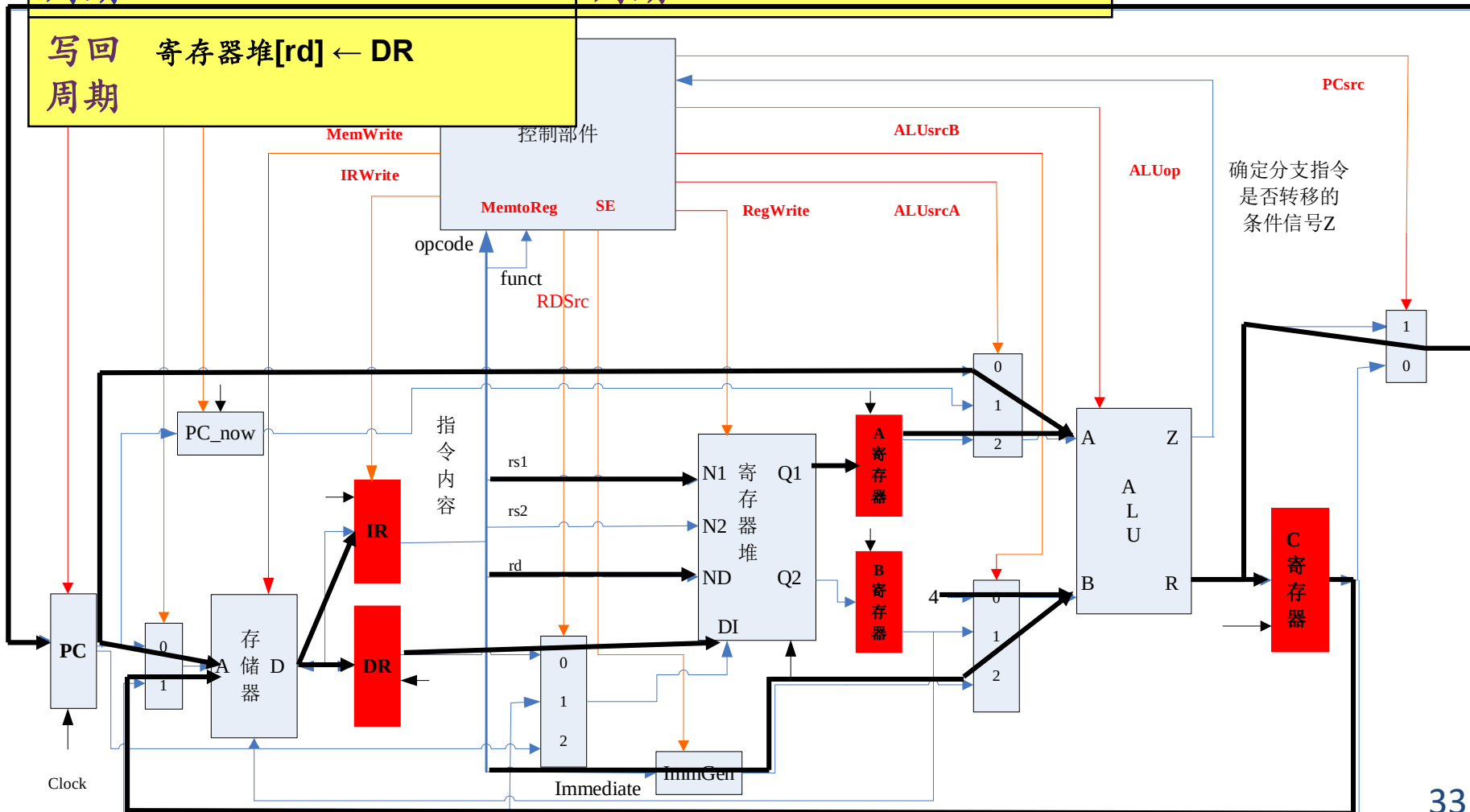


RISC-V 的 I 型 LW 指令的执行过程

LW

I-type

取指周期	$IR \leftarrow MEM[PC]$ $PC \leftarrow PC+4$	译码周期	$A \leftarrow [rs1]$
执行周期	$C \leftarrow A + \text{扩展 Imm}$	内存周期	$DR \leftarrow MEM[C]$
写回周期	寄存器堆[rd] $\leftarrow DR$		

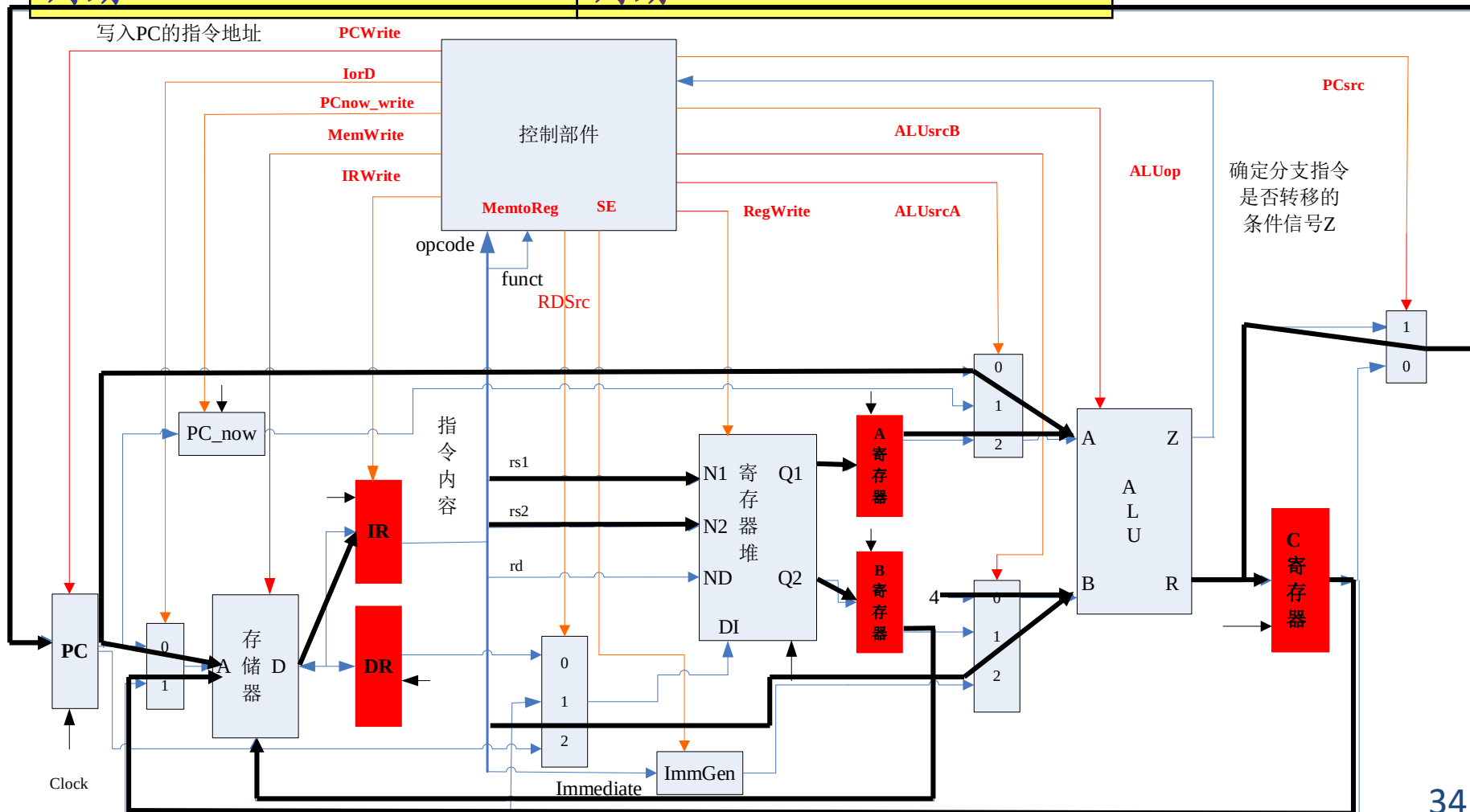


RISC-V 的 S 型 SW 指令的执行过程

SW

S-type

取指 周期	$IR \leftarrow MEM[PC]$ $PC \leftarrow PC+4$	译码 周期	$A \leftarrow [rs1]$ $B \leftarrow [rs2]$
执行 周期	$C \leftarrow A + \text{扩展 Imm}$	内存 周期	$MEM[C] \leftarrow B$



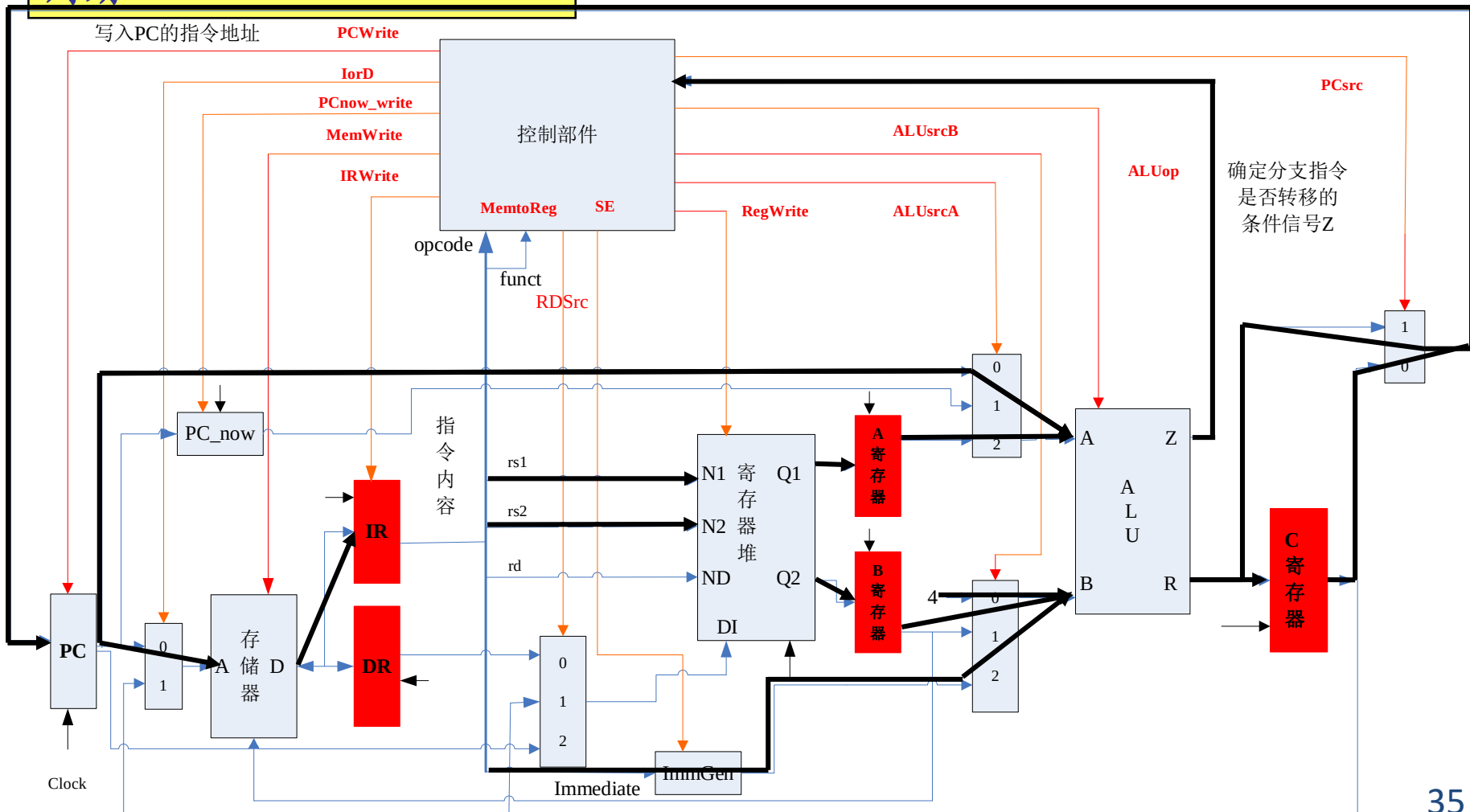
RISC-V 的 B 型 BEQ 指令的执行过程

BEQ
B-type

取指周期
 $IR \leftarrow MEM[PC]$
 $PC_now \leftarrow PC, PC \leftarrow PC+4$

译码周期
 $A \leftarrow [rs1] \quad B \leftarrow [rs2]$
 $C \leftarrow PC_now + B \text{ 扩展 Imm}$

执行周期
 $(A-B) == 0 ? PC \leftarrow C : \text{nop}$



RISC-V 的 J 型 JAL 指令的执行过程

JAL

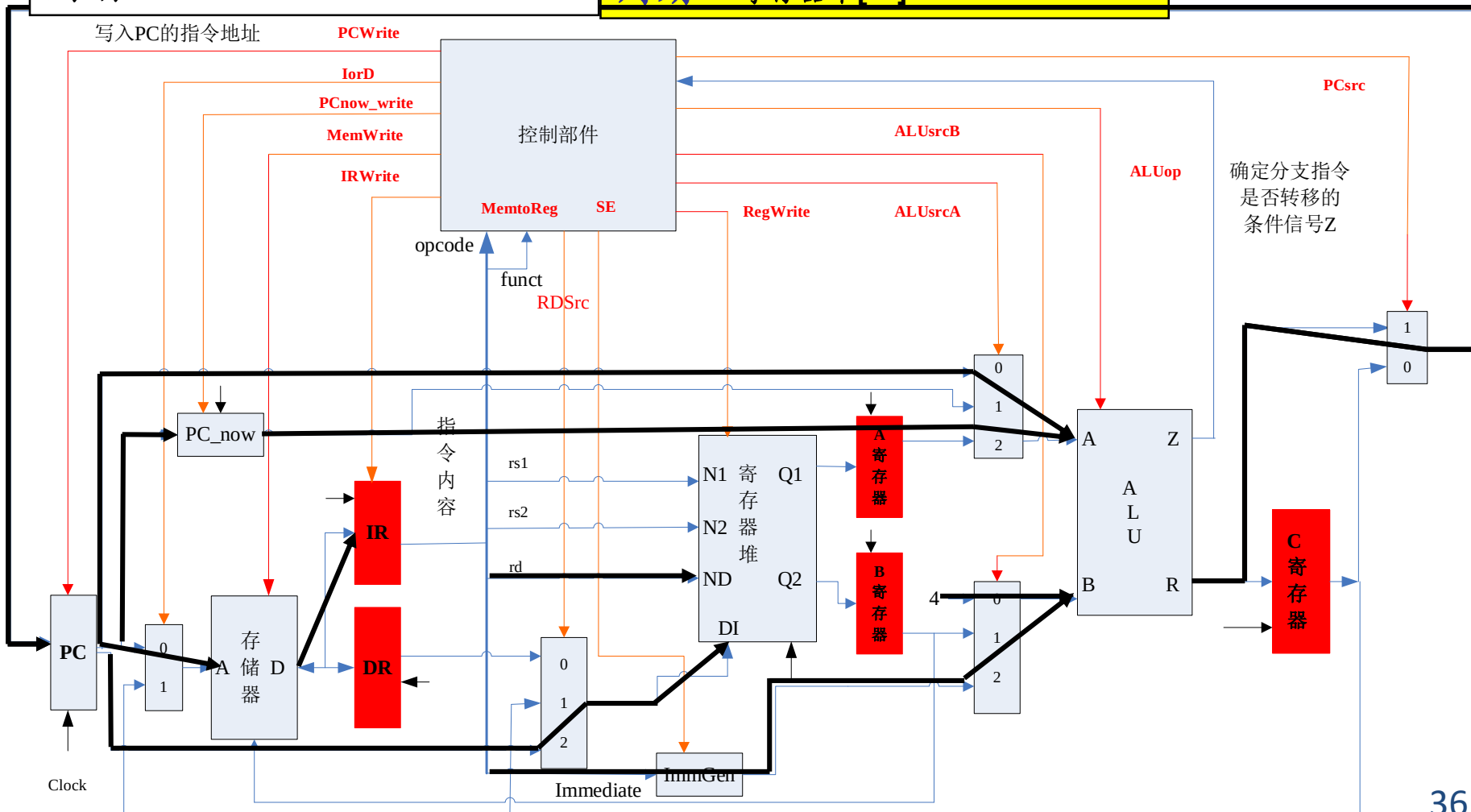
J-type

取指周期
 $IR \leftarrow MEM[PC]$
 $PC_now \leftarrow PC, PC \leftarrow PC+4$

译码周期
 $A \leftarrow [rs1] \quad B \leftarrow [rs2]$
 $C \leftarrow PC_now + B \text{ 扩展 Imm}$

执行周期
 $C \leftarrow PC_now + J \text{ 扩展 Imm}$

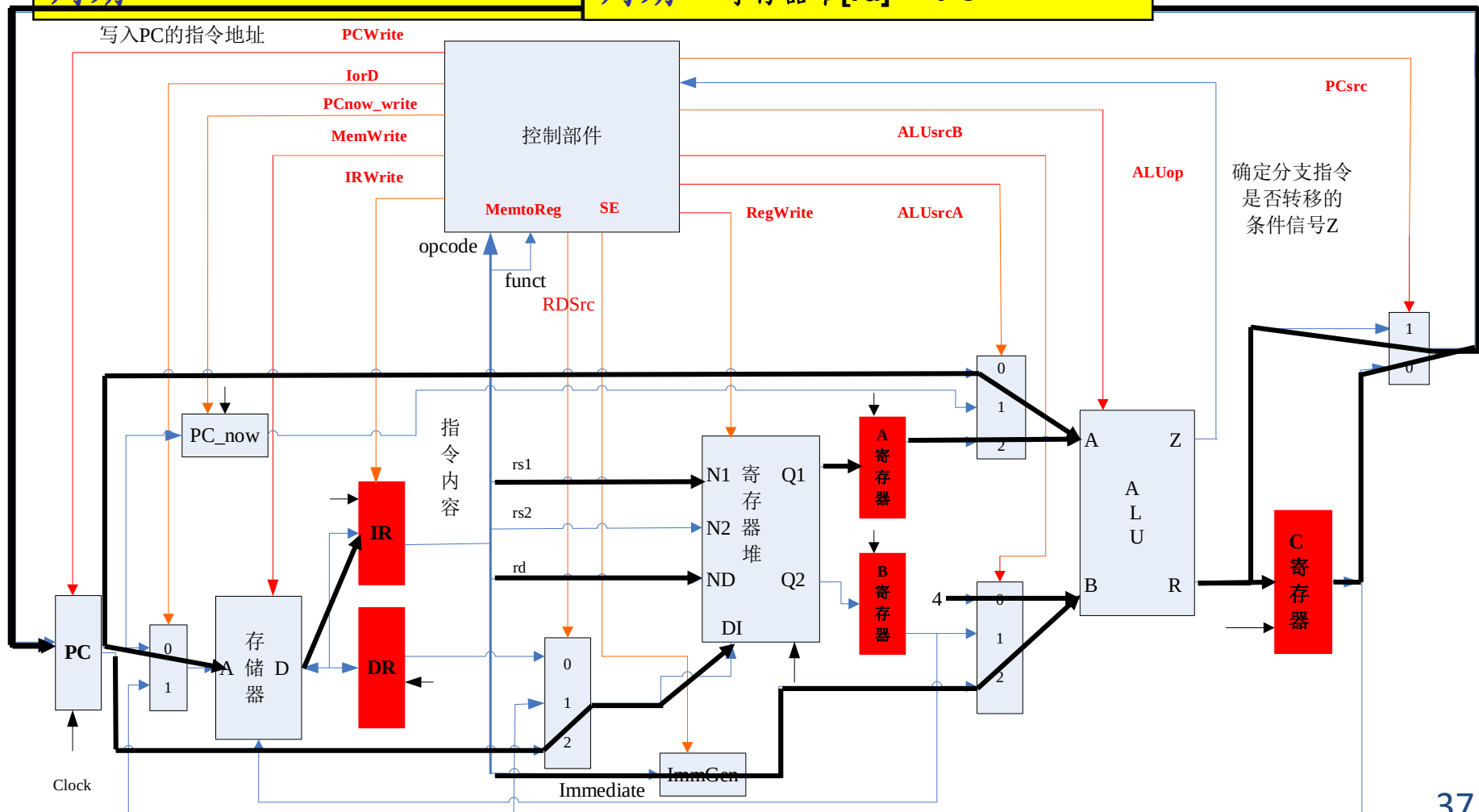
写回周期
 $PC \leftarrow C$
寄存器堆[rd] $\leftarrow PC$



RISC-V 的 I 型 JALR 指令的执行过程

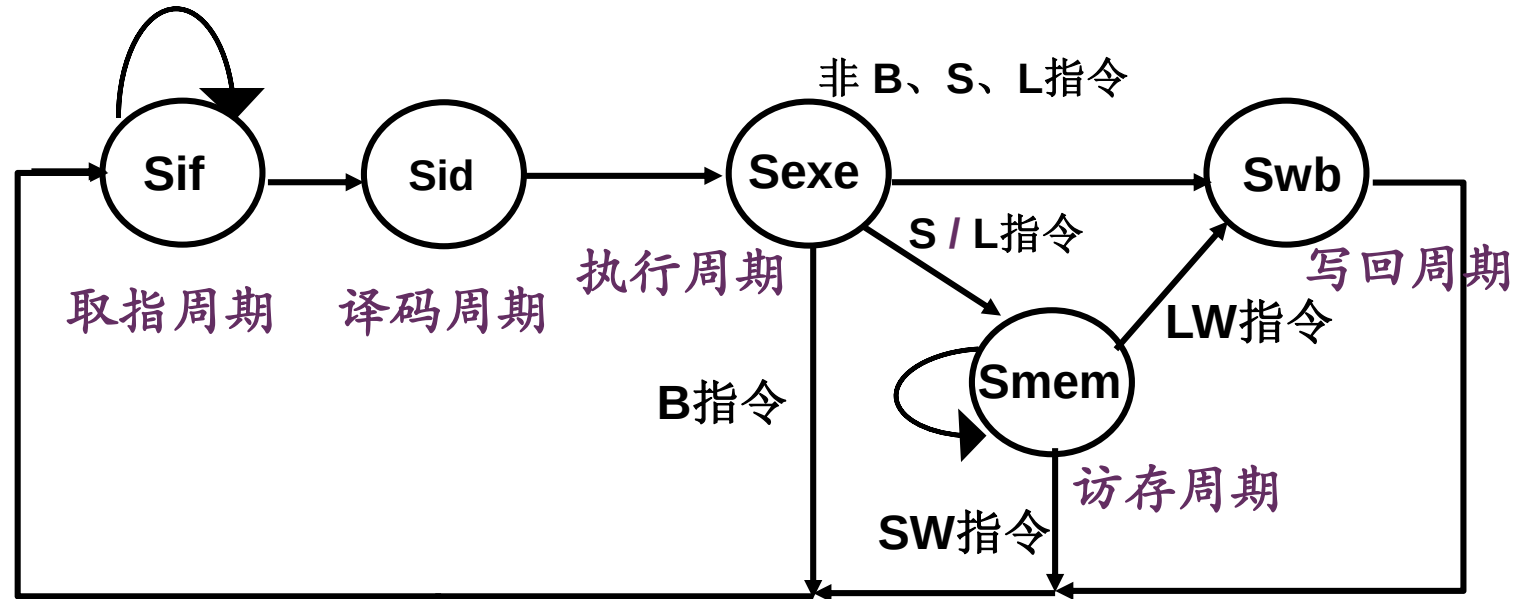
JALR
I-type

取指周期 $IR \leftarrow MEM[PC]$ $PC \leftarrow PC+4$	译码周期 $A \leftarrow [rs1]$ $B \leftarrow [rs2]$ $C \leftarrow PC_now + B \text{ 扩展 Imm}$
执行周期 $C \leftarrow A + I \text{ 扩展 Imm}$	写回周期 $PC \leftarrow C$ 寄存器堆[rd] $\leftarrow PC$



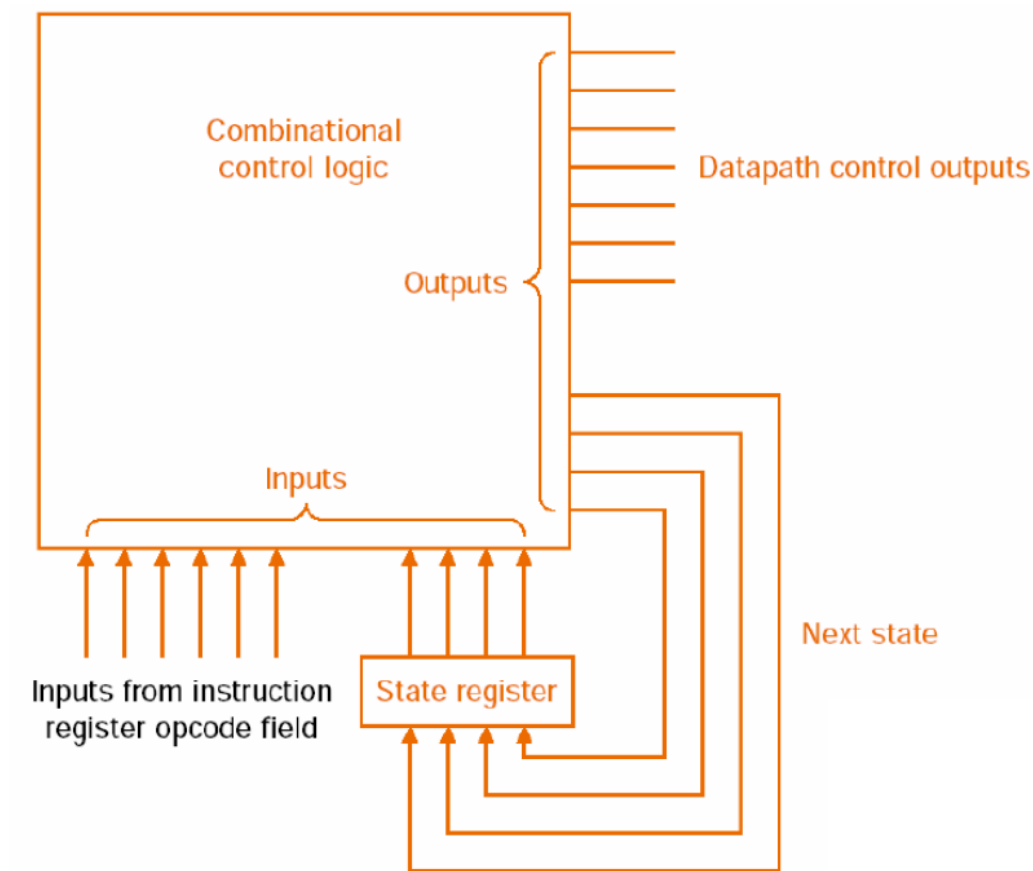
AUIPC U-type

状态转移图和指令各执行步骤的操作功能



指令步骤	读取指令	指令译码	执行运算	内存读写	数据写回
B 指令	$IR \leftarrow MEM[PC]$ $PC \leftarrow PC + 4$	$C \leftarrow PC_now + imm$	若条件成立 则 $PC \leftarrow C$		
R/I 运算		$A \leftarrow Reg[rs1]$	$C \leftarrow A \text{ op } B$		$Reg[rd] \leftarrow C$
S 指令			$C \leftarrow A + \text{各类符号扩展}(Imm)$	$Mem[C] \leftarrow B$	
L 指令				$DR \leftarrow Mem[C]$	$Reg[rd] \leftarrow DR$
Jal指令	$PC_now \leftarrow PC$	$B \leftarrow Reg[rs2]$			$Reg[rd] \leftarrow PC$

FSM Implementation



使用有限状态机标志执行步骤

状态机代码-1

```
// generate next FSM state given by `fsm_state_reg`, `op` and `bus.ack`
always_ff @(posedge clk) begin: fsm_update
    if (rst) begin
        fsm_state_reg <= FETCH;
    end else begin
        unique case (fsm_state_reg)
        FETCH: begin
            if (bus.ack) fsm_state_reg <= DECODE;
            else fsm_state_reg <= fsm_state_reg; // hold when waiting for ack
        end
        DECODE: begin
            fsm_state_reg <= EXECUTE;
        end
        EXECUTE: begin
            if (opcode == OP_CODE_LOAD || opcode == OP_CODE_STORE) begin
                fsm_state_reg <= ACCESS;
            end else begin
                if (gpr_we) fsm_state_reg <= COMMIT; //goto COMMIT stage if write GPR
                else fsm_state_reg <= FETCH;
            end
        end
    end
end
```


状态机代码-2

```
ACCESS: begin
    if (bus.ack) begin
        if (gpr_we) fsm_state_reg <= COMMIT;
        else fsm_state_reg <= FETCH;
    end else begin
        fsm_state_reg <= fsm_state_reg; // hold when waiting for ack
    end
end
default: begin
    fsm_state_reg <= FETCH;
end
endcase
end
end
```

小结

□ 多周期CPU

- 按指令的执行步骤，每个步骤占用一个CPU周期
- 不同指令的指令周期不同
- 指令串行执行
- 提高了整体性能

□ 进一步的改进

- 各部件的利用率依然偏低
 - 指令并行执行？是否可行？如何进行？

阅读和思考

□ 阅读

- 有关参考书

□ 思考

- 多周期CPU的特点，并与单周期CPU比较
- 如何进一步提高性能？
- 如何实现指令流水？

谢谢

计算机硬件系统组成

